

台灣自編大學教科書

@prof

機器人學導論：機構、感知、控制與具身人工智慧

第一篇 機器人的身體與系統

第 3 章

機器人的組成與系統架構

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】一台被完全拆解的服務型機器人「解剖照」：鋁擠型骨架、兩顆帶減速機的馬達、主控板與驅動板、光達與相機、鋰電池、線束與紅色急停按鈕，分門別類地攤放在工作檯上，旁邊各以標籤註明其在系統中的角色——骨骼、肌肉、大腦、感官、心臟、神經、保險。

叫獸開講 「機器人有骨骼、肌肉、神經和大腦嗎？」

實驗室的解剖檯上，躺著一台壽終正寢的服務型機器人。它曾經在某家醫院的走廊送藥，服役三年，最後因為主機板燒毀而退役，被廠商捐給了叫獸的實驗室。今天下午的課，主題只有一個：把它拆開。

學生們圍成一圈，帶著手套與螺絲起子。第一層外殼卸下，露出一根鋁擠型的骨架，上面鑽滿了螺絲孔。「這是它的骨骼，」叫獸說，「決定它有多重、多硬、能承載多少東西。」接著是兩顆帶減速機的直流馬達，齒輪箱上還沾著三年份的灰塵。「這是肌肉。注意看，馬達本身轉得很快但沒有力氣，靠減速機把轉速換成扭力——就像人的肌肉配上槓桿一樣的骨頭。」

再往裡，一片主機板與四張小電路板現身。「這裡是大腦與脊髓。」叫獸指著中間那片：「這是主控電腦，跑地圖與路徑規劃，一秒鐘想十次。旁邊這四片是馬達驅動器，一秒鐘想一千次——它們負責把大腦說的『往前走』翻譯成每個輪子每毫秒該轉多快。」

有位學生舉手：「為什麼要分兩層？一台電腦全部做完不就好了？」叫獸拿起一條被燒黑的排線：「因為它們的時間尺度差一百倍。大腦可以偶爾卡頓零點一秒，你不會發現；但如果馬達控制迴路卡頓零點一秒，機器人就會撞牆。這叫『上位機與下位機』的分工，是所有機器人系統的共同架構。」

拆到最後，只剩一顆鋰電池、幾捆線與一個紅色的蘑菇頭按鈕。學生把零件在桌上排成一排：骨架、馬達、感測器、控制板、電池、線束、急停開關——七堆。叫獸環顧一圈：「現在請告訴我，這台機器人是怎麼死的？」

學生們翻找燒毀的主機板，卻在電池旁邊發現了答案：一條電源線的絕緣皮被機構的邊角磨破了，長期摩擦後短路，燒穿了電源模組，連帶燒毀主機板。「三年的研發、幾十萬的成本，」叫獸把那條線舉起來，「死於一條沒有加保護套管的電線。這就是為什麼我們要用一整章來談『系統』——因為機器人不是零件的加總，而是零件之間關係的總和。任何一個環節的疏忽，都能讓其他所有的精妙設計歸零。」

本章為什麼重要

第 1 章談了機器人是什麼，第 2 章談了怎麼學，從本章開始進入實質內容。本章的任務是建立「系統觀」——在深入每一個子系統（第 4~7 章的機構、馬達、感測器、電子）之前，先看

清楚它們如何組合成一台完整的機器人。這個順序很重要：如果先學馬達再學系統，你會知道很多零件卻不知道怎麼組；先建立系統觀，每學一個零件你都知道它在整體中的位置與責任。本章同時建立三個貫穿全書的重要概念：感知—規劃—控制的軟體分層、上位機與下位機的硬體分工、以及即時與非即時運算的時間尺度差異。這三個概念會在第 29 章的 ROS 2、第 34 章的產線整合中反覆出現。

學習目標

- 能列舉機器人系統的七大子系統，並說明各自的功能與相互關係。
- 能繪製一台機器人的系統方塊圖，標示訊號流與功率流。
- 能區分控制器、嵌入式電腦與微控制器的角色差異與選型考量。
- 能說明感測器系統的訊號鏈路：物理量、轉換、調理、取樣到數值。
- 能計算機器人的功率需求與電池續航時間。
- 能比較常見通訊介面的頻寬、距離與適用場合。
- 能說明感知—規劃—控制三層架構的資料流與各層時間尺度。
- 能解釋即時系統的意義，並判斷哪些任務必須即時、哪些不必。
- 能說明上位機與下位機的分工原則，並舉出實際的硬體組合。
- 能比較集中式與分散式機器人架構的優缺點與適用時機。

先備知識

第 1 章的感知—思考—行動循環概念。本章涉及少量電學計算（歐姆定律、功率與電池容量），若不熟悉可先閱讀附錄 D.1 的基本電學速成，或在閱讀 3.5 節時再回頭補充。本章不需要任何微積分或矩陣運算。

核心名詞中英對照

表 3-1 本章核心名詞

中文	英文	說明
子系統	Subsystem	構成完整系統的功能單元
機械本體	Mechanical Structure	機器人的骨架、連桿與外殼

中文	英文	說明
控制器	Controller	執行控制演算法、輸出指令的運算單元
微控制器	Microcontroller Unit, MCU	整合處理器與周邊介面的單晶片
單板電腦	Single Board Computer, SBC	具作業系統的小型電腦，如 Raspberry Pi
驅動器	Driver	將控制訊號放大為馬達所需功率的電路
訊號調理	Signal Conditioning	放大、濾波、隔離感測器訊號的處理
類比數位轉換器	Analog-to-Digital Converter, ADC	將連續電壓轉為數值的元件
取樣頻率	Sampling Rate	每秒讀取訊號的次數
即時系統	Real-Time System	必須在規定時限內完成回應的系統
控制週期	Control Cycle	控制迴路重複執行一次的時間間隔
上位機	Host Computer / High-Level Controller	負責規劃與決策的高階運算單元
下位機	Low-Level Controller	負責即時控制與硬體介面的運算單元
現場匯流排	Fieldbus	工業設備間的即時通訊網路，如 CAN、EtherCAT
系統方塊圖	System Block Diagram	以方塊與箭頭表示子系統與訊號流的圖

3.1 機器人的機械本體

3.1.1 骨架的三個任務

機械本體是機器人的骨骼與軀殼，承擔三個任務。第一是支撐：承受自身重量、負載重量與運動時的慣性力。第二是定位：保證各個關節與感測器的相對位置精確且穩定——如果相機的安裝支架會晃動，再好的視覺演算法也算不準物體位置。第三是保護：外殼隔絕粉塵、水氣與碰撞，同時避免人員接觸到運動件與高壓電。

這三個任務常常互相衝突。要支撐得穩就要粗壯，但粗壯就重；要保護周全就要密封，但密封就散熱困難。機器人的機構設計，本質上是在剛性、重量、成本與可維護性之間找平衡點——第 4 章與第 27 章會深入這個主題。

3.1.2 常見的結構形式與材料

實務上常見的結構形式有：鋁擠型骨架（模組化、易改裝，教學與研發首選）、鈹金外殼

(量產成本低、剛性佳)、鑄件(大型工業手臂的關節座，剛性最高但開模成本高)、碳纖維(輕量高剛性，用於無人機與競賽機器人)、以及 3D 列印件(快速原型與異形零件)。

材料的選擇有一個容易被忽略的關鍵指標：比剛度(剛度除以密度)。對移動機器人與機械手臂而言，增加的每一克重量都要消耗能量去搬運，並降低可用負載——這就是為什麼賽車與機器人都愛用鋁與碳纖維，而不是更便宜的鋼。

3.1.3 一個新手常犯的錯：忽略累積誤差

學生自製機器手臂時，常常每一節都做得「差不多」，結果末端偏差達數公分。原因是誤差會沿著運動鏈累積：第一關節的安裝偏差零點五度，經過三十公分的連桿放大後，末端就偏了將近三公釐；三個關節加起來，誤差可能達到一公分以上。這個現象在第 9 章的正向運動學中會有數學上的解釋——連桿越長、關節越多，對安裝精度的要求越高。

◎ 拆解練習的價值

如果你的實驗室或社團有報廢的印表機、掃地機器人或影印機，請務必拆開它。你會看到：工程師如何用最便宜的方式達成定位(塑膠齒輪加光遮斷器)、如何佈線避免磨損(套管與束帶)、如何做防呆(接頭形狀不同就插不進去)。這些細節在教科書裡佔不到半頁，卻是量產工程的精華。

3.2 控制器與嵌入式電腦

3.2.1 三個層級的運算單元

機器人的「大腦」通常不是一顆晶片，而是一組分工的運算單元。依運算能力與即時性由低到高排列，可分為三個層級。

微控制器(MCU)：如 Arduino 使用的 ATmega、STM32、ESP32。特點是內建類比數位轉換器、PWM 輸出、UART / I²C / SPI 介面，開機即執行程式，沒有作業系統的延遲，時序極為穩定。適合做馬達控制迴路、感測器讀取、安全監控。運算能力有限(通常數十至數百 MHz、數十至數百 KB 記憶體)，無法跑影像辨識。

單板電腦(SBC)：如 Raspberry Pi、樹莓派同級產品。具備完整的 Linux 作業系統，可跑 Python、OpenCV 與 ROS 2，能處理影像、地圖與網路。缺點是作業系統會造成不可預測的延遲(排程、記憶體管理)，不適合直接做毫秒級的控制迴路。

邊緣 AI 運算平台：如 NVIDIA Jetson 系列，在單板電腦的基礎上加入 GPU 或專用 AI 加速

器，能即時執行深度學習模型（物件偵測、語意分割、VLA 模型）。功耗與成本較高，通常只在需要視覺 AI 的機器人上。

表 3-2 三個層級運算單元的比較

層級	代表產品	運算能力	即時性	典型任務	本書章節
微控制器 MCU	Arduino、 STM32、ESP32	低（MHz 級）	極佳（無 OS 延遲）	馬達控制、感測器讀 取、急停監控	第 7、16 章
單板電腦 SBC	Raspberry Pi 5	中（GHz 級 CPU）	普通（OS 排 程延遲）	路徑規劃、SLAM、 ROS 2 節點	第 25、29 章
邊緣 AI 平 台	NVIDIA Jetson 系 列	高（含 GPU / NPU）	普通	影像辨識、深度學習 推論、VLA	第 22、31 ~ 33 章

3.2.2 選型的三個問題

面對琳瑯滿目的開發板，選型只需回答三個問題。第一，最嚴苛的即時要求是多少？如果需要一毫秒的控制迴路，就必須有 MCU。第二，最重的運算任務是什麼？如果要跑 YOLO 物件偵測，就需要 GPU 或 NPU。第三，開發時間有多少？MCU 的開發較繁瑣但穩定，SBC 上用 Python 開發極快但需處理作業系統的複雜性。

多數實用的機器人採用組合方案：SBC 或 Jetson 當上位機負責規劃與感知，MCU 當下位機負責即時控制，兩者用 UART 或 CAN 連接。這正是 3.8 節要展開的主題，也是開頭故事中那台送藥機器人的架構。

3.3 感測器系統

3.3.1 訊號鏈路：從物理世界到程式變數

感測器不是插上去就有數字。一個完整的感測器系統包含五個環節，任何一環出問題，程式讀到的都是垃圾。

第一是換能：把物理量轉成電訊號。應變片把力轉成電阻變化、光電二極體把光轉成電流、編碼器把角度轉成脈波。第二是訊號調理：微弱的訊號需要放大，高頻雜訊需要濾除，高壓或不同接地的訊號需要隔離。第三是類比數位轉換（ADC）：把連續的電壓轉成數值，解析度（幾位元）決定了能分辨多細的差異。第四是取樣：以固定頻率重複讀取，取樣頻率必須至少是訊號最高頻率的兩倍（Nyquist 定理），否則會產生混疊失真。第五是數值處理：單位換算、校正、濾

波，最後才成為程式裡那個叫做 distance 或 angle 的變數。

圖 3-1 感測器訊號鏈路

建議配圖：由左至右五個方塊——物理量（力／光／角度）→ 換能器 → 訊號調理（放大、濾波、隔離）→ ADC 與取樣 → 數值處理（校正、濾波）→ 程式變數。下方標註各環節常見的問題：靈敏度不足、雜訊、解析度不夠、混疊、單位錯誤。

3.3.2 內部感測與外部感測

如第 1 章所述，感測器分為兩大類。內部感測告訴機器人「我自己怎麼了」：關節角度（編碼器）、姿態與角速度（IMU）、馬達電流（間接反映負載與碰撞）、電池電壓。外部感測告訴機器人「外面有什麼」：距離（超音波、紅外線、光達）、影像（相機、深度相機）、力與接觸（力覺感測器、觸覺開關）、位置（GNSS、UWB 室內定位）。

一個關鍵的工程原則是：內部感測是控制的基礎，外部感測是自主的基礎。沒有編碼器，馬達連轉了幾度都不知道，連 PID 都做不了；沒有外部感測，機器人只能重播固定動作，無法應對環境變化。這也解釋了為什麼工業手臂可以只有編碼器（環境固定），而移動機器人必須有豐富的外部感測（環境未知）。

3.3.3 感測器融合的必要性

沒有任何單一感測器是完美的。編碼器精確但會累積誤差（輪子打滑時它一無所知）；IMU 反應快但會漂移；GNSS 不漂移但在室內失效且更新慢；相機資訊豐富但怕光線變化。實務上的解法是融合：用多個互補的感測器彼此校正。輪速計提供短期精確的位移、IMU 提供快速的姿態變化、光達提供絕對的環境參照，三者融合後的定位精度遠優於任何單一來源。這正是第 24 章 Kalman 濾波器的核心價值。

3.4 致動器與驅動器

3.4.1 致動器：能量轉換的終點

致動器把電能（或液壓、氣壓的能量）轉換成機械運動。機器人上最常見的是電動致動器：直流馬達（便宜、控制簡單）、無刷直流馬達（效率高、壽命長、功率密度大）、步進馬達（開迴路即可定位、低速扭力大）、伺服馬達（內建位置回授，直接下角度指令）。液壓致動器功率密度極高，用於大型工程機械與早期的人形機器人；氣壓致動器則柔順、便宜，常用於夾爪。第 5 章會完整比較。

3.4.2 驅動器：被忽略的關鍵環節

控制器輸出的是幾毫安培的訊號，馬達需要的是幾安培甚至數十安培的電流——中間必須有驅動器 (Driver) 做功率放大。這是初學者最常忽略、卻最常燒毀的環節。

驅動器的核心是功率電晶體組成的 H 橋電路，透過 PWM 訊號控制導通比例，決定馬達的等效電壓與轉速；改變橋臂的導通組合，則決定轉向。選用驅動器時必須注意三個規格：持續電流（能長時間承受的電流）、峰值電流（啟動或堵轉瞬間的電流，通常是持續電流的數倍）、以及散熱能力。學生常見的失敗是：只看馬達的額定電流選驅動器，忽略了堵轉電流可能是額定值的五到十倍，結果第一次撞牆就燒掉驅動器。

3.4.3 致動器與機構的匹配

馬達不是越大越好。選型時要同時考慮：所需扭力（含加速時的慣性扭力，不只是靜態負載）、所需轉速、減速比（減速機把高轉速低扭力換成低轉速高扭力）、以及慣量匹配（馬達轉子慣量與負載慣量相差太大時，控制會不穩定或反應遲鈍）。這幾項的計算會在第 5 章展開，本章只需建立一個觀念：致動器、減速機與機構是一個整體，不能分開選。

3.5 電源、電池與電源管理

3.5.1 為什麼電源是機器人的第一殺手

在教學現場，機器人「動不了」的原因排行榜上，電源問題長年高居第一。原因是電源同時牽涉三個容易出錯的面向：電壓不對（燒毀元件或帶不動）、電流不足（一啟動就重開機）、以及接地不當（訊號亂跳、通訊中斷）。而且電源問題的症狀往往偽裝成別的問題——程式看起來當機、感測器讀值突然亂跳、通訊莫名斷線，追查半天才發現是電池電壓在馬達啟動時瞬間掉到臨界值以下。

3.5.2 常見電池與關鍵規格

機器人常用的電池有三類。鋰聚合物電池 (LiPo) 放電電流大、能量密度高，是無人機與競賽機器人的主流，但過充過放會膨脹起火，必須使用專用充電器與平衡充。鋰離子電池 (Li-ion ，如 18650) 能量密度高、循環壽命長，適合續航需求高的移動機器人。鉛酸電池笨重但便宜且耐用，多見於大型 AGV 與工業設備。

選電池時要看四個規格：標稱電壓（決定能不能直接驅動馬達）、容量（以 mAh 或 Ah 表

示，決定續航）、放電倍率（C 數，決定能瞬間輸出多大電流）、以及重量。一個常被忽略的陷阱是放電倍率：一顆 2000 mAh、10C 的電池最大可放 20 A，但同樣容量 1C 的電池只能放 2 A——後者接上大電流馬達，電壓會瞬間崩塌。

3.5.3 電源架構：分離與穩壓

良好的機器人電源架構有兩個原則。第一是分離：功率電源（馬達、加熱器）與訊號電源（微控制器、感測器）盡量分開供電，或至少在電源入口分開走線，最後單點共地。這能避免馬達的電流突波拉低控制器電壓。第二是穩壓：電池電壓會隨放電下降（鋰電池從 4.2 V 降到 3.0 V），必須經過降壓（Buck）或升壓（Boost）模組穩壓後，才供給需要穩定電壓的元件。

此外還需要保護電路：過流保護（保險絲）、過放保護（低電壓切斷）、逆接保護（防止電池接反），以及反電動勢保護（馬達停止瞬間會產生反向電壓，需以二極體或電容吸收）。這些元件加起來成本不到兩百元，卻能保住價值數萬元的主控板。

圖 3-2 機器人電源架構示意圖

建議配圖：電池 → 主開關與保險絲 → 分為兩路：上路經降壓模組供給控制器與感測器（訊號電源，5 V / 3.3 V），下路直接供給馬達驅動器（功率電源）；兩路在電池負極單點共地。急停開關串接於功率電源路徑上。

3.5.4 續航估算

續航時間的粗略估算方式為：續航（小時）= 電池容量（Ah）× 可用比例 ÷ 平均消耗電流（A）。可用比例通常取 0.8（保留 20% 避免過放）。平均消耗電流需分項估算：主控電腦、感測器、馬達（依工作週期加權）。例題 3-1 會完整示範這個計算。

實務上有兩個經驗法則。第一，馬達的平均消耗遠低於峰值，但電源系統必須按峰值設計。第二，實測續航通常比計算值短 20% 至 30%，因為電池老化、低溫、與轉換效率損失。設計時務必留有餘裕。

3.6 通訊系統

3.6.1 機器人內部的三層通訊

一台機器人內部的通訊可分為三層。晶片級：微控制器與感測器晶片之間，使用 I²C（兩線、可掛多個裝置、速度較慢）或 SPI（四線、速度快、適合高速感測器如 IMU）。板級：控制板之間，使用 UART（簡單的點對點序列通訊）或 CAN Bus（多節點、抗雜訊強、有優先權機制，汽

車與工業標準)。系統級：上位機與外部世界之間，使用乙太網路、WiFi 或 5G。

工業機器人與高階系統則使用現場匯流排：EtherCAT (基於乙太網的即時協定，可達微秒級同步，適合多軸協調)、PROFINET、EtherNet/IP。這些協定的共同特徵是「確定性」——不只是快，而是每次傳輸的延遲都可預測，這對多軸同步至關重要。

表 3-3 常見通訊介面比較

介面	線數	速度	距離	多節點	典型用途
I ² C	2	100k ~ 1M bps	數十公分	可 (位址定址)	感測器、EEPROM
SPI	4+	數 M ~ 數十 Mbps	數十公分	可 (片選)	高速 IMU、SD 卡、顯示器
UART	2	9.6k ~ 1M bps	數公尺	否 (點對點)	模組通訊、除錯訊息
CAN Bus	2	125k ~ 1 Mbps	數十至數百公尺	可 (廣播 + 優先權)	多馬達驅動器、車輛
乙太網路	4 ~ 8	100M ~ 1 Gbps	百公尺	可 (交換機)	相機、光達、上位機
EtherCAT	4 ~ 8	100 Mbps (微秒級同步)	百公尺	可 (串接)	工業多軸即時控制
WiFi / 藍牙	無線	數 M ~ 數百 Mbps	數十公尺	可	遙控、監控、資料回傳

3.6.2 選擇通訊介面的三個考量

選介面時考慮三件事。第一是頻寬需求：一顆 IMU 每秒送幾百筆小資料，用 I²C 綽綽有餘；一台深度相機每秒送數十 MB，就必須用 USB 3 或乙太網路。第二是即時性需求：控制指令必須準時抵達，適合 CAN 或 EtherCAT；影像串流可以容忍偶爾延遲，用 WiFi 即可。第三是抗雜訊能力：機器人內部有大電流馬達，長距離的訊號線容易受干擾，CAN 的差動訊號比 UART 的單端訊號可靠得多。

◎ 無線通訊的陷阱

許多學生把 WiFi 用在控制迴路上——遙控時感覺良好，比賽時卻因為現場數百支手機的干擾而失控。原則是：安全相關與即時控制絕不依賴無線；無線只用於監控、遙測與非即時的指令下達。急停功能更必須是硬體線路，不能透過任何通訊協定。

3.7 軟體與演算法

3.7.1 機器人軟體的三層堆疊

機器人的軟體同樣分層。最底層是韌體與驅動層：直接操作硬體暫存器，讀取編碼器、輸出 PWM、處理中斷。中間層是控制與狀態估測層：PID 迴路、運動學計算、感測器融合、里程計。最上層是規劃與決策層：路徑規劃、任務排程、行為決策、人機介面。

這種分層有兩個好處。第一是可替換性：換一顆不同的馬達，只需改韌體層，上層演算法不動。第二是可測試性：每一層可以獨立驗證——先確認編碼器讀值正確，再測試 PID 是否收斂，最後才測整體路徑追蹤。跳過分層測試而直接跑全系統，是初學者最常見的除錯災難。

3.7.2 中介框架：為什麼需要 ROS 2

當機器人的軟體模組超過五個，就會面臨一個問題：模組之間如何交換資料？自己寫通訊機制不僅費時，還很難支援跨語言（C++ 與 Python 混用）與跨機器（上位機與遠端電腦）。ROS 2 正是為此而生的中介框架：它提供節點（獨立的程式模組）、話題（發布訂閱式的資料流）、服務（請求回應式的呼叫）、動作（有進度回報的長時間任務）等機制，讓模組化開發變得標準化。

此外，ROS 2 的生態系提供了大量現成套件：SLAM、路徑規劃、感測器驅動、視覺化工具，可以直接站在巨人的肩膀上。第 29 章會完整介紹，本章只需理解它在系統中的定位——它是軟體模組之間的「神經系統」。

3.7.3 狀態機：機器人行為的骨架

除了演算法，機器人軟體還需要一個行為的組織方式，最常見的是有限狀態機（Finite State Machine）。以送藥機器人為例，狀態可能包括：待命、前往取藥點、等待裝載、前往病房、等待取藥、返回充電。每個狀態有明確的進入條件、執行內容與離開條件。狀態機的價值在於行為變得可預測、可測試、可除錯——當機器人卡住時，第一個問題永遠是「它現在在哪個狀態？」

3.8 上位機與下位機

3.8.1 為什麼要分工

開頭故事中學生的疑問——「為什麼不用一台電腦全部做完」——答案是時間尺度。控制迴路需要每一毫秒執行一次且不能延遲，這是即時任務；路徑規劃可能需要幾十毫秒計算一次，偶爾慢一點無妨，這是非即時任務。把兩者放在同一個作業系統上，作業系統的排程隨時可能為了處理網路封包而打斷控制迴路，導致馬達失控。

因此標準做法是硬體上分家：上位機（SBC 或工控機，跑 Linux 與 ROS 2）負責感知、地圖、規劃與人機介面；下位機（MCU，裸機或即時作業系統）負責馬達控制迴路、感測器高頻取樣、安全監控。兩者以 UART、CAN 或乙太網路連接，上位機下達「速度指令」，下位機回傳「編碼器與狀態」。

圖 3-3 上位機與下位機的分工架構

建議配圖：上半部為上位機（Jetson / Raspberry Pi）方塊，內含 SLAM、路徑規劃、視覺辨識、ROS 2 節點；下半部為下位機（STM32 / Arduino）方塊，內含 PID 控制迴路、編碼器讀取、急停監控。中間以雙向箭頭連接，上行標註「速度指令（10~50 Hz）」，下行標註「里程計與狀態回報（100~1000 Hz）」。下位機再連接馬達驅動器與感測器。

3.8.2 介面設計的原則

上下位機之間的介面設計，有三個實務原則。第一是抽象層級要一致：上位機應該說「以每秒零點五公尺前進、每秒零點二弧度左轉」，而不是「左輪 PWM 給 180、右輪給 150」——後者把機構細節洩漏到了上層。第二是要有超時保護：如果上位機當機或連線中斷，下位機必須在數百毫秒內自動停止馬達，而不是維持最後一個指令繼續衝出去。第三是狀態要雙向：下位機不只回傳感測值，也要回傳自身狀態（是否過流、是否急停、電池電壓），讓上位機能做出正確決策。

3.8.3 典型的硬體組合

依規模不同，常見的組合有：入門級——ESP32 單機兼任上下位（適合循跡小車，因為運算需求低）；中階——Raspberry Pi 5 加 STM32 或 Arduino，以 UART 連接（適合教學型 AMR）；高階——NVIDIA Jetson 加多顆馬達驅動器，以 CAN 或 EtherCAT 連接（適合研究型移動機器人與機械手臂）；工業級——工控機加專用運動控制卡，以 EtherCAT 連接伺服驅動器（適合產線設備）。

3.9 即時控制與非即時運算

3.9.1 什麼是即時

「即時」(Real-Time) 不等於「快」，而是「準時」。一個即時系統的定義是：不僅要算出正確答案，還必須在規定的時限 (deadline) 之前算出來，否則答案就失去價值。一個平均延遲一毫秒但偶爾延遲一百毫秒的系統，不是即時系統；一個穩定延遲五毫秒的系統，反而是。

即時性又分兩級。硬即時 (Hard Real-Time)：錯過時限會造成災難，例如手臂的力矩控制迴路、無人機的姿態控制、急停響應。軟即時 (Soft Real-Time)：錯過時限只降低品質，例如影像顯示掉幀、地圖更新慢一拍。分清楚哪些任務是硬即時，是系統設計的核心決策。

3.9.2 典型的時間尺度

機器人系統中，不同層級的控制週期有數量級的差異，理解這個「時間金字塔」對系統設計極為重要。電流迴路 (馬達驅動器內部)：約 10 ~ 100 微秒。速度與位置迴路 (下位機)：約 0.1 ~ 1 毫秒。運動學與軌跡插補：約 1 ~ 10 毫秒。狀態估測與定位：約 10 ~ 100 毫秒。路徑規劃：約 100 毫秒至數秒。任務決策與 AI 推論：數百毫秒至數秒。

這個金字塔解釋了許多設計選擇。為什麼馬達驅動器要有自己的處理器？因為 10 微秒的迴路無法交給上層。為什麼視覺不能直接進控制迴路？因為相機每秒三十幀 (33 毫秒)，比控制迴路慢了兩個數量級——這正是視覺伺服控制 (第 22 章) 的核心難題。

表 3-4 機器人系統的時間尺度金字塔

層級	典型週期	執行位置	即時性要求	範例任務
電流迴路	10 ~ 100 μ s	馬達驅動器	硬即時	PWM 換相、電流控制
速度 / 位置迴路	0.1 ~ 1 ms	下位機 MCU	硬即時	PID 控制、編碼器讀取
軌跡插補	1 ~ 10 ms	下位機或上位機	硬即時	運動學計算、軌跡生成
狀態估測	10 ~ 100 ms	上位機	軟即時	感測器融合、里程計、定位
路徑規劃	0.1 ~ 數秒	上位機	非即時	A*、RRT、避障重規劃
任務決策 / AI	數百 ms ~ 數秒	上位機 / 邊緣 AI	非即時	物件辨識、任務排程、LLM

3.9.3 如何達成即時性

達成即時性有三個層次的做法。最簡單的是裸機程式 (bare-metal)：MCU 上沒有作業系統，用計時器中斷觸發控制迴路，時序最精確，適合單一控制任務。其次是即時作業系統 (RTOS，如 FreeRTOS)：提供多任務與優先權排程，高優先權任務可搶佔低優先權任務，適

合任務較多的下位機。最複雜的是即時 Linux (PREEMPT_RT 補丁)：在完整的 Linux 上取得毫秒級的可預測性，適合需要同時跑 ROS 2 與即時控制的工控機。

此外還有一個常被忽略的實務要點：即時任務中絕對不能做「執行時間不可預測」的事——不要動態配置記憶體、不要印出除錯訊息、不要等待網路回應。這三件事是初學者破壞即時性的三大元凶。

3.10 感知—規劃—控制架構

3.10.1 三層架構的資料流

把前面各節整合起來，機器人的軟體系統呈現一個經典的三層架構：感知層、規劃層、控制層。這是第 1 章「感知—思考—行動」在工程實作上的具體化。

感知層負責回答「世界現在是什麼樣子」。它接收原始感測資料（影像、點雲、編碼器脈波、IMU 讀值），經過濾波、融合與特徵擷取，輸出一個結構化的世界模型：我在地圖上的哪個座標、周圍有哪些障礙物、目標物在哪裡。這一層的產出是「狀態」與「地圖」。

規劃層負責回答「我該做什麼」。它接收感知層的世界模型與任務目標，輸出一條可執行的軌跡或一串動作序列。這一層包含全域路徑規劃（從 A 到 B 怎麼走）、區域避障（前面突然出現行人怎麼繞）、以及任務層決策（先送藥還是先充電）。

控制層負責回答「怎麼讓身體照做」。它接收規劃層的軌跡，計算每個關節或輪子該輸出多少力矩或速度，並透過回授不斷修正誤差。這一層直接與硬體對話，也是即時性要求最高的一層。

圖 3-4 感知—規劃—控制三層架構

建議配圖：三個橫向色帶由上而下——感知層（相機、光達、IMU、編碼器 → 融合 → 狀態與地圖）、規劃層（任務目標 + 地圖 → 全域路徑 → 區域避障 → 軌跡）、控制層（軌跡 → 運動學 → PID → 驅動器 → 馬達）。右側標註各層的更新頻率；左側以一條回饋箭頭由馬達經環境回到感測器，形成閉環。

3.10.2 各層的時間尺度與耦合

三層的更新頻率遞增：規劃層最慢（0.1 ~ 數秒）、感知層居中（10 ~ 100 毫秒）、控制層最快（0.1 ~ 1 毫秒）。這個梯度是自然的——世界的變化沒有那麼快，但馬達的物理響應非常快。

但三層並非完全獨立。當感知層發現前方突然出現障礙物時，必須能快速觸發規劃層重新規

劃，甚至直接觸發控制層緊急煞車——這條「反射弧」通常繞過規劃層，直接由感知連到控制，就像人類碰到燙的東西會先縮手再思考。這種設計在機器人學中稱為反應式行為（reactive behavior），與深思熟慮的規劃式行為（deliberative behavior）互補。

3.10.3 架構的演變：從三層到端到端

值得注意的是，這個經典三層架構近年正受到挑戰。深度學習與 VLA 模型的興起，帶來了「端到端」（end-to-end）的可能性：直接把相機影像輸入神經網路，輸出馬達指令，中間不再有明確的地圖與規劃層。兩種路線各有優劣：三層架構可解釋、可除錯、可驗證安全性；端到端架構能處理難以建模的複雜情境，但像個黑盒子。目前的主流看法是混合——用學習處理感知與技能，用傳統架構保證安全與可靠。第 33 章會深入這個辯論。

3.11 集中式與分散式機器人架構

3.11.1 集中式：一個大腦說了算

集中式架構把所有運算集中在一台主控電腦上：它讀取所有感測器、執行所有演算法、輸出所有指令，周邊只有簡單的訊號轉換電路。優點是設計簡單、資料同步容易、除錯集中。缺點是：布線複雜（所有感測器的線都要拉到主控）、可靠性差（主控當機則全機癱瘓）、擴充困難（增加一顆馬達可能就得換主控板）。小型教學機器人多採此架構。

3.11.2 分散式：各司其職的節點網路

分散式架構把運算分散到多個節點：每顆馬達有自己的智慧驅動器（內含電流與速度迴路）、每組感測器有自己的處理模組，彼此以 CAN 或 EtherCAT 連接，主控只負責高層協調。優點是布線簡潔（一條匯流排串接所有節點）、可靠性高（單一節點故障可隔離）、易於擴充（新增節點只需接上匯流排並設定位址）。缺點是成本較高、需要處理通訊協定與時間同步。工業機器人、汽車與大型移動平台幾乎都採此架構。

表 3-5 集中式與分散式架構的比較

面向	集中式	分散式
運算位置	單一主控	多個智慧節點
布線	星狀，線束複雜	匯流排，串接簡潔
可靠性	低（單點故障即全癱）	高（可隔離故障節點）
擴充性	差（受主控 I/O 限制）	佳（掛上匯流排即可）

面向	集中式	分散式
成本	低	較高
除錯難度	低（狀態集中）	較高（需觀察匯流排）
典型應用	教學機器人、小型專題	工業機器人、AGV、汽車

3.11.3 混合式與實務選擇

實務上多數機器人採混合式：關鍵的即時控制分散到智慧驅動器，感知與規劃集中在一台上位機。選擇的原則是——依「軸數」與「距離」決定。軸數少（三軸以下）、距離短（同一塊板子上）採集中式即可；軸數多（六軸以上）、距離長（分佈在整台車上）則必須分散式。第 34 章討論產線整合時，會看到這個原則如何延伸到整個工廠的層級。

3.12 機器人的系統方塊圖

3.12.1 方塊圖的價值

系統方塊圖是機器人工程師的共同語言。它用方塊代表子系統、用箭頭代表訊號或功率的流向，讓整台機器人的架構在一頁紙上一目了然。它的價值有三：溝通（讓機構、電子與軟體工程師對齊認知）、設計（在動手前發現介面缺口與衝突）、除錯（故障時能快速定位到某個方塊或某條連線）。

3.12.2 畫方塊圖的五個步驟

步驟一，先畫出四個核心方塊：感測器、控制器、驅動器、致動器，並以箭頭連成閉環（別忘了從致動器經「環境」回到感測器的那條回饋線）。步驟二，加入電源系統，用不同顏色或粗線標示功率流，與訊號流區分開來。步驟三，若採上下位機架構，將控制器方塊拆成上位機與下位機，標註兩者的通訊介面與方向。步驟四，在每條連線上標註介面規格（I²C、CAN、PWM）與更新頻率。步驟五，補上安全相關元件：急停開關、保險絲、過流保護，並確認急停能直接切斷功率而不經過軟體。

圖 3-5 教學型自主移動機器人系統方塊圖範例

建議配圖：完整方塊圖。左側感測器群（光達、深度相機、IMU、編碼器）；中央上位機（Raspberry Pi 5：SLAM、路徑規劃、ROS 2）以 UART 連下位機（STM32：PID、里程計、急停監控）；右側馬達驅動器與兩顆直流馬達。下方電池經開關、保險絲、急停後分為功率路（粗線至驅動器）與訊號路（經降壓模組至上下位機與感測器），單點共地。各連線標註介面與頻率。

3.12.3 由方塊圖到規格表

方塊圖畫完後，下一步是把每個方塊與連線轉成規格表：每個元件的型號、電壓、電流、通訊介面、更新頻率、成本。這份表格就是採購清單與設計文件的基礎，也是第 36 章系統工程流程的起點。學生專題最常見的失敗，就是跳過這一步直接買零件——結果買回來才發現感測器是 5 V 訊號、主控是 3.3 V，或是驅動器的電流撐不住馬達的堵轉。

完整計算例題

例題 3-1 電池續航估算

題目：一台教學型 AMR 使用 12 V、10 Ah 的鋰電池。系統耗電如下：主控電腦（Raspberry Pi 5 含光達）平均 1.5 A；下位機與感測器 0.3 A；兩顆驅動馬達在行進時各 2.0 A，但工作週期（行進時間佔比）為 60%。取電池可用比例 80%。求理論續航時間。若實測續航為 1.8 小時，求折損率並分析可能原因。

解：先計算平均消耗電流。馬達平均電流 = $2 \text{ 顆} \times 2.0 \text{ A} \times 0.6 = 2.4 \text{ (A)}$ 。總平均電流 = $1.5 + 0.3 + 2.4 = 4.2 \text{ (A)}$ 。可用電量 = $10 \text{ Ah} \times 0.8 = 8 \text{ (Ah)}$ 。理論續航 = $8 \div 4.2 \approx 1.90 \text{ (小時)}$ ，約 1 小時 54 分。

實測 1.8 小時，折損率 = $(1.90 - 1.80) \div 1.90 \approx 5.3\%$ ，屬於合理範圍。可能原因包括：降壓模組的轉換效率（通常 85~92%，會多耗約 10% 的電）、電池老化導致實際容量低於標稱、地毯或斜坡使馬達電流高於估算值、以及低溫下鋰電池內阻上升。若折損率超過 20%，則應懷疑有異常耗電（如驅動器待機電流過高）或電池已嚴重老化。

例題 3-2 控制週期與取樣頻率設計

題目：某差速輪機器人的輪子編碼器每圈輸出 1000 個脈波，輪子直徑 10 公分，最高速度 1.0 m/s。（a）求最高速度時的脈波頻率。（b）若下位機的控制週期為 1 ms，每個週期能計數幾個脈波？此解析度對應的速度誤差為何？（c）若改用 10 ms 控制週期，速度解析度如何變化？

解：（a）輪子周長 = $\pi \times 0.1 \approx 0.314 \text{ (m)}$ 。最高速度時每秒轉數 = $1.0 \div 0.314 \approx 3.18 \text{ (轉/秒)}$ 。脈波頻率 = $3.18 \times 1000 \approx 3183 \text{ (Hz)}$ 。

（b）1 ms 內的脈波數 = $3183 \times 0.001 \approx 3.18 \text{ (個)}$ 。由於計數必須是整數，實際只會讀到 3 個或 4 個，量化誤差達 ± 1 個脈波。一個脈波對應的距離 = $0.314 \div 1000 = 0.314 \text{ (mm)}$ ，

在 1 ms 內對應的速度解析度 = $0.000314 \div 0.001 = 0.314$ (m/s) ——這高達最高速度的 31% , 完全無法用於精確的速度控制。

(c) 改用 10 ms 週期，脈波數 ≈ 31.8 個，量化誤差仍為 ± 1 個脈波，但速度解析度變為 $0.000314 \div 0.01 = 0.0314$ (m/s)，僅為最高速度的 3.1%，改善了十倍。

結論與工程啟示：控制週期越短，控制反應越快，但速度解析度越差——這是一組必須權衡的矛盾。實務解法有三：一是提高編碼器線數（如改用 4000 線）；二是採用邊緣計時法（測量脈波之間的時間間隔而非計數）；三是採用串級控制，內層電流迴路用高頻、外層速度迴路用較低頻。這個「快與準」的權衡，會在第 16 章的數位控制中再次出現。

程式範例：系統健康監控

以下 Python 程式模擬一個機器人的系統監控模組：定期檢查各子系統的狀態，並在異常時分級告警。這是每一台實用機器人都應具備的基本功能——開頭故事中那台送藥機器人，若有電源異常監控，或許就不會因為一條磨破的電線而報廢。

```
import time
import random

# 各子系統的安全門檻
LIMITS = {
    "battery_v": (11.0, 12.6), # 電池電壓 (V) : 低於 11.0 需警告
    "motor_a": (0.0, 4.0), # 馬達電流 (A) : 超過 4.0 疑似堵轉
    "cpu_temp": (0.0, 75.0), # 主控溫度 (°C)
    "comm_delay": (0.0, 50.0), # 上下位機通訊延遲 (ms)
}

def read_sensors():
    """模擬讀取各子系統狀態（實機上改為真實感測器讀值）"""
    return {
        "battery_v": random.uniform(10.5, 12.6),
        "motor_a": random.uniform(0.5, 5.0),
        "cpu_temp": random.uniform(45.0, 80.0),
        "comm_delay": random.uniform(5.0, 60.0),
    }

def check_health(readings):
    """檢查狀態，回傳（等級，訊息）清單"""
    alerts = []
    for key, value in readings.items():
        lo, hi = LIMITS[key]
        if value < lo:
```

```

        alerts.append(("嚴重", f"{key} 過低：{value:.1f} ( 下限 {lo} ) "))
    elif value > hi:
        level = "嚴重" if value > hi * 1.1 else "警告"
        alerts.append((level, f"{key} 過高：{value:.1f} ( 上限 {hi} ) "))
    return alerts

def main_loop(cycles=5):
    for i in range(cycles):
        readings = read_sensors()
        alerts = check_health(readings)
        print(f"--- 第 {i+1} 次檢查 ---")
        if not alerts:
            print("所有子系統正常")
        for level, msg in alerts:
            print(f"[{level}] {msg}")
            if level == "嚴重":
                print(" → 動作：停止馬達、進入安全狀態、回報上位機")
        time.sleep(0.2)

main_loop()

```

請試著修改：加入一個「連續三次警告才升級為嚴重」的邏輯，避免單次雜訊造成誤停機。這個「去彈跳」的概念，在真實的安全監控系統中極為重要。

ROS 2 與模擬實作：畫出你的系統方塊圖

本章的實作不寫程式，而是設計。請依以下步驟，為你打算製作的機器人（或附錄中的任一）繪製完整的系統方塊圖與規格表：

1. 在紙上或繪圖軟體（建議 draw.io 或 Fritzing）畫出四個核心方塊：感測器、控制器、驅動器、致動器，並連成閉環。
2. 加入電源系統：電池、開關、保險絲、降壓模組、急停開關。以粗線標示功率流、細線標示訊號流，並標出單點共地位置。
3. 決定是否採用上下位機架構。若是，將控制器拆為兩個方塊並標註通訊介面（UART / CAN）與方向。
4. 在每條連線上標註介面規格與更新頻率，例如「I²C, 100 Hz」「PWM, 20 kHz」「UART 115200, 50 Hz」。
5. 將方塊圖轉為規格表：元件名稱、型號、工作電壓、額定電流、通訊介面、預估單價，並加總成本。

6. 檢查三件事：（一）所有元件的電壓是否相容？（二）電源的總電流是否足夠（含峰值）？（三）急停是否直接切斷功率、不經過軟體？

（進階）若已安裝 ROS 2，可執行 `rqt_graph` 觀察一個現成範例的節點圖——你會發現它就是本章方塊圖的軟體版本。第 29 章會教你如何為自己的系統建立這張圖。

國中小也看得懂：機器人的身體和我們一樣

機器人就像我們的身體，也有好幾個「系統」在一起工作。骨頭是機器人的鋁架，撐住身體不讓它垮掉。肌肉是馬達，負責出力讓輪子轉、手臂動。眼睛耳朵是感測器，看得到前面有沒有東西、知道自己轉了幾圈。大腦是控制板，負責想「該往哪走」。血管是電線，把電池的電送到每個地方。神經是訊號線，把眼睛看到的東西告訴大腦，再把大腦的命令傳給肌肉。

最重要的一件事：這些系統要「合作」才有用。如果眼睛很好但大腦沒接到線，機器人還是會撞牆；如果大腦很聰明但電池沒電，它就一動也不動。所以做機器人時，不能只顧一個部分做得很棒，要讓每個部分都健康、而且好好連在一起。這就叫做「系統」。下次你的小車不會動，記得照順序檢查：有電嗎？線接好了嗎？眼睛有看到嗎？大腦有想嗎？肌肉有動嗎？

叫獸提醒與常見錯誤

- 誤區一：先買零件再想架構。應該先畫方塊圖、列規格表，確認電壓與介面相容後再採購——順序反了會浪費大量金錢與時間。
- 誤區二：只看馬達額定電流選驅動器。堵轉電流可能是額定的五到十倍，驅動器必須按峰值選，並加上過流保護。
- 誤區三：訊號線與功率線網在一起走。馬達的電流突波會透過電磁耦合干擾訊號線，導致感測器讀值亂跳。兩者應分開走線或加隔離。
- 誤區四：把急停做成軟體功能。急停必須是硬體迴路，直接切斷功率電源。任何經過軟體或通訊協定的「急停」，在系統當機時都會失效。
- 誤區五：把即時控制放在跑作業系統的電腦上。Linux 的排程延遲不可預測，毫秒級控制迴路請交給 MCU 或即時作業系統。
- 誤區六：忽略通訊超時保護。上位機當機時，下位機若沒有超時停止機制，機器人會維持最後指令一路衝出去。

- 誤區七：不做分層測試。請依序驗證：電源正常 → 感測器讀值正確 → 單軸控制收斂 → 多軸協調 → 全系統。跳級測試會讓除錯時間暴增。
- 誤區八：布線不做保護。開頭故事的教訓——線束經過機構邊角處必須加套管、固定與圓角處理，這是最便宜的可靠度投資。

課堂討論

1. 開頭故事中的送藥機器人死於一條磨破的電線。若你是設計者，會用哪三項具體措施防止此事？請分別從機構、電子與軟體監控三方面提出。
2. 為什麼掃地機器人可以用集中式架構，而六軸工業手臂幾乎都用分散式？請從軸數、距離與可靠度三方面分析。
3. 端到端的 AI 控制（影像直接輸出馬達指令）省略了明確的規劃層。你認為在什麼場合可以接受？什麼場合絕對不行？
4. 如果你只能為機器人增加一項「監控」功能來提升可靠度，你會選什麼？請說明理由與實作方式。
5. 比較人體與機器人的系統架構：人類的「下位機」在哪裡（提示：脊髓反射）？這個對照對機器人設計有什麼啟發？

章末摘要

- 機器人由七大子系統組成：機械本體、控制器、感測器、致動器與驅動器、電源、通訊、軟體；系統的價值不在零件加總，而在關係的總和。
- 運算單元分三層：MCU（即時性佳、運算低）、SBC（可跑 OS 與 ROS 2）、邊緣 AI 平台（可跑深度學習）；實用機器人多採組合方案。
- 感測器訊號鏈路有五環：換能、訊號調理、ADC、取樣、數值處理；任一環出錯，程式讀到的都是垃圾。
- 驅動器必須按峰值電流（堵轉可達額定五至十倍）選型，並具備過流與散熱設計。
- 電源是機器人故障排行榜第一名；架構原則為功率與訊號分離、單點共地、加裝穩壓與保護電路；續航估算須保留 20 ~ 30% 餘裕。
- 通訊介面依頻寬、即時性與抗雜訊選擇；安全相關與即時控制絕不依賴無線，急停必須是硬

體迴路。

- 軟體分三層：韌體驅動、控制與狀態估測、規劃與決策；ROS 2 是模組間的神經系統，有限狀態機是行為的骨架。
- 上下位機分工的理由是時間尺度差異；介面設計三原則為抽象層級一致、超時保護、狀態雙向回報。
- 即時不等於快，而是準時；系統存在時間尺度金字塔，由電流迴路（微秒）到 AI 決策（秒），跨越六個數量級。
- 感知—規劃—控制三層架構是第 1 章循環的工程實作；集中式適合小型系統，分散式適合多軸長距離；系統方塊圖是工程師的共同語言。

觀念題與計算題

觀念題

1. 列舉機器人的七份子系統，並各用一句話說明其功能。
2. 比較 MCU、SBC 與邊緣 AI 平台的差異，並各舉一個適合的任務。
3. 畫出感測器的五環訊號鏈路，並在每一環標出一個常見的失效模式。
4. 為什麼說「內部感測是控制的基礎，外部感測是自主的基礎」？請以工業手臂與移動機器人各舉一例說明。
5. 說明驅動器在系統中的角色，以及選型時必須注意的三個規格。
6. 電源架構的兩大原則是什麼？為什麼馬達啟動會導致微控制器重開機？
7. 比較 I²C、CAN Bus 與 EtherCAT 三種介面的特性與適用場合。
8. 解釋「即時不等於快」的意義，並區分硬即時與軟即時，各舉兩個機器人上的例子。
9. 說明上位機與下位機分工的理由，以及介面設計的三個原則。
10. 比較集中式與分散式架構的優缺點，並說明選擇的判準。
11. 說明感知—規劃—控制三層架構的資料流，以及「反射弧」為何要繞過規劃層。
12. 畫系統方塊圖的五個步驟為何？為什麼急停必須畫在功率路徑上而非訊號路徑上？

計算題

1. 某巡檢機器人使用 24 V、20 Ah 電池。耗電為：主控 2.0 A、光達 0.5 A、四顆馬達各 1.5 A

(工作週期 70%)、雲台 0.3 A (工作週期 20%)。電池可用比例取 80%，降壓模組效率 90%。求理論續航時間 (提示：訊號側耗電需除以轉換效率)。

2. 某機械手臂關節使用 2500 線編碼器 (每圈 2500 脈波)，減速比 50:1，馬達最高轉速 3000 rpm。(a) 求關節端的最高轉速 (度 / 秒)。(b) 求馬達端最高脈波頻率。(c) 若控制週期為 0.5 ms，計算關節角度的量化解析度 (度)。(d) 若要求角度解析度優於 0.01 度，編碼器線數至少需多少？
3. 某移動機器人採 CAN Bus 連接 6 個節點，每個節點每 10 ms 送出一則 8 位元組的訊息 (連同協定開銷共約 130 位元)。(a) 求匯流排的平均使用率 (以 500 kbps 計)。(b) 若要新增 4 個相同節點，使用率變為多少？是否仍在建議的 40% 安全上限內？

動手做與專題延伸

本章實作：機器人解剖與系統還原

找一台報廢或閒置的電子設備 (掃地機器人、噴墨印表機、光碟機、玩具遙控車皆可)，在安全前提下拆解，並完成下表。拆解前請先斷電並取出電池；若含高壓元件 (如印表機的電源模組、CRT) 請勿拆解。

表 3-6 解剖記錄表 (範例列供參考)

子系統	找到的零件	規格 / 型號	在系統中的角色	設計巧思或缺陷
機械本體	塑膠底盤與金屬支架	ABS 塑膠、厚約 2 mm	支撐與保護內部元件	以卡榫取代螺絲，組裝快但難維修
致動器	直流馬達加塑膠齒輪箱	6 V、減速比約 1:48	驅動輪子前進與轉向	齒輪為塑膠，成本低但易磨損
(自行填寫其餘五個子系統)				

完成後，請依 3.12 節的五步驟，把你拆到的零件還原成一張系統方塊圖，並回答：這台設備採用集中式還是分散式架構？它有沒有上下位機的分工？

專題延伸

- 為附錄 F 的循跡小車繪製完整系統方塊圖與規格表，並計算其電池續航時間。實測後比較理論值與實測值的差異，分析原因。

- 訪查一家自動化設備廠商或參觀工廠，記錄一台工業機器人的系統架構：主控是什麼？用什麼匯流排？急停如何設計？安全圍籬與感測如何配置？寫成一千字的參訪報告。
- 進階：為一台假想的「校園導覽機器人」做完整的系統設計——列出需求（續航四小時、能上下斜坡、能辨識人臉、能語音互動），據此選定各子系統的元件、繪製方塊圖、估算成本與續航，並標出三個最大的技術風險與因應方案。

參考文獻與延伸閱讀

- Siciliano, B. and Khatib, O. (Eds.), Springer Handbook of Robotics, 2nd ed., Springer (第一部分：機器人系統與元件) 。
- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson.
- Corke, P., Robotics, Vision and Control, 3rd ed., Springer.
- Niku, S. B., Introduction to Robotics: Analysis, Control, Applications, Wiley (機器人子系統與致動器章節) 。
- Horowitz, P. and Hill, W., The Art of Electronics, 3rd ed., Cambridge University Press (電源、訊號調理與雜訊處理) 。
- Kopetz, H., Real-Time Systems: Design Principles for Distributed Embedded Applications, Springer (即時系統理論) 。
- ROS 2 官方文件 (docs.ros.org)：節點、話題與系統架構設計指南。
- CAN in Automation (CiA) 與 EtherCAT Technology Group 官方技術文件。
- IEC 60204-1 與 ISO 13850：機械電氣安全與緊急停止裝置相關標準。
- 各開發板官方文件：Raspberry Pi、NVIDIA Jetson、STM32、ESP32 資料手冊與應用筆記。